

Outsmarting the projectiles: A Drone Simulation for Evading Bullets and Explosives

Thomas van der Sterren
thvan23@student.sdu.dk
University of Southern Denmark

Adam Kolodziej
adkol23@student.sdu.dk
University of Southern Denmark

Abstract—In this study, the development of a drone simulation was made, with the capabilities to avoid countermeasures. The goal of this simulation is to understand possible vulnerabilities in existing countermeasure systems to help strengthen the defense capabilities. Throughout the development, Matlab Simulink was used to represent a drone model that was flown in a virtual environment. At the end it became clear that creating an avoidance system is not only possible, but should be something that countermeasure development should keep in mind.

Index Terms—UAV, UAS, Drone, Evading Maneuvers, Enemy Avoidant, Matlab

I. INTRODUCTION

The modern battlefield has adapted to become a 3 dimensional field, where UAVs, commonly known as drones, have grown to become a larger component. These versatile platforms have already been a part of larger militias and military operations. Nevertheless the threat is increasing with the coming of smaller, faster and easy-to-use drones accessible for anyone. To maintain a safe airspace, more anti-drone techniques are being developed and already deployed. Though these techniques might still be lacking the abilities that new drones might exploit. To aid the development of the countermeasure techniques, it is of importance to understand what possible threats are and how they could be implemented to counteract the current preventive measures. Although there are a lot of types of drones, within this study, small agile drones have been examined, as this kind of drone is becoming a larger threat within a fast developing battlefield. Within this study, the main driver is to assess how drones could behave under countermeasure influences and try to avoid them. The knowledge gained within this study could then be used and applied to the procedures and preventive measures used.

II. BACKGROUND

A. Red Teaming

Red teaming, in the terms of military development, is a process to simulate how hostile/enemy forces might approach tactics, a new weapons system or

their strategy. In principle it consists of creating a team of experts which act as the opposing force and challenge potential weaknesses within newly developed strategies or technological advancements.

According to the University of Foreign Military and Cultural Studies (UFMCS) there are four main principles that come to play when Red Teaming. The four principles are: Self-awareness and Reflection, Groupthink Mitigation (GTM) and Decision Support, Fostering Cultural Empathy, Applied Critical Thinking. These four principles help guide the Red Team to critically think as an opposing force. [1]

As a part of Red Teaming, developers and researchers sometimes develop or work on projects that seem very inhumane or extreme. While their development never sees final stages, it does give the ability to understand the possibilities for a hostile force. Thus such research is often still undertaken and explored to find vulnerabilities in existing technologies.

Additionally some research is seen as good developments of own systems, and can be implemented as such.

B. Current Battlefield

Drones have been used on the battlefield for a long time. It goes back to the early 90's during the Gulf War. However, the usage of drones has dramatically changed over time. When analyzing current battlefields, such as that of Ukraine, it becomes apparent that the term drone is a flexible tool. During the war in Ukraine, two classes of drones are used. Being Class I and Class III drones. The difference in these classes come from the size and usage of the drones. The Class III drones used, such as Turkish Bayraktar TB2, are mainly used for larger strikes and surveillance from high altitude. This kind of drone usages was also seen in earlier conflicts such as in the middle-east. What has increased in the war in Ukraine is the usage of Class I drones. These drones are small, user-friendly and affordable. These drones have gained immense popularity during this conflict. The drones are primarily used for real-time reconnaissance and short-range attacks, done by strapping small explosives to it. They have also

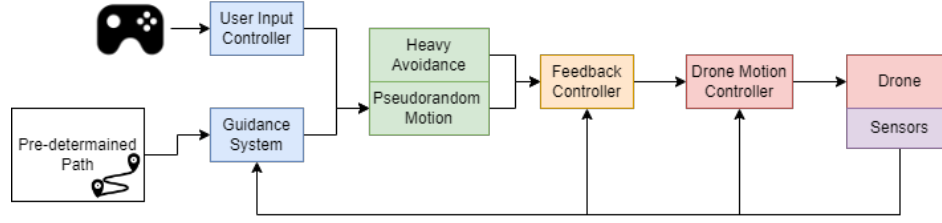


Figure 1. Schematic view of interaction between systems.

been seen to do small supply drops to troops that are located in difficult to reach areas. Although most of these advances are not revolutionary, but rather evolutionary it does bring into perspective what the future of warfare can hold. Where drones are more common, and more accessible for use by the small troops at the front-line. [2]

C. Damaging Projectiles

As the drone threat has become more apparent, finding ways to counter them has also increased. Drones pose a very big security risk, not only during military operations, but also for civil security. Mitigating these risks is thus an objective that leads a large field of research. Early methods of mitigation drones are mainly focused on detection and disruption, where existing jamming technologies were adapted to account for drone frequencies. These jamming systems are now small and lightweight that can easily be mounted on vehicles or around bases. However, not all infantry will walk with these, thus shooting down drones with small caliber is something still done. As drones become more self-sufficient, jamming signals would not be enough. That is why the defensive market sees weapon systems able to detect and take down these drones, one example would be that of Rheinmetall Oerlikon weapon as part of their skynet security system. [3].

Designing drones that are able to withstand such complex security systems is something that can be expected to be the next step. Like so, next to autonomy to surpass jamming systems, drones that aim to surpass security systems will be agile to move around small caliber fire arms, and evasive for larger anti-drone systems.

D. Running Tactics

Like many professionals in the field of defense state, moving in an unpredictable manner can save your life. It is common knowledge that when your movement is predictable, a good shooter can adjust their aim to react accordingly. For many military's, it is therefore taught to move unpredictably or for very short moments in a specific direction. Reducing the risk of getting hit. This not only holds true for personnel but also for aerial systems. Many anti-air

defense systems work based on prediction of targets. A lot of research is also being done on the development of new predictive models. [4] Therefore the key element to work on is becoming unpredictable.

III. METHOD

This chapter will describe how the setup was developed and implemented for testing.

A. System Design

In order for the drone to be small caliber arms avoidant, random movement patterns need to be introduced as well as a large explosive avoidant system. Looking at figure 1, a schematic overview is presented of the drone design. The controlling of the drone is done by either a controller, or a multi-coordinate path which it should follow. Before the direction is acted out by the drone controller, a pseudo-random noise is generated. The random noise is generated using a Gaussian distribution. Additionally this block also has a 'heavy avoidance' connection. This is a block that takes over control completely and upon activation will cause the drone to immediately shoot out of its current location. The goal being to avoid an in-air explosive projectile. This information together with the sensor data is processed by the controller after which the drone motion controller will process the data for the drones actuation. The drone is equipped with sensors to get information about its current angle, and position. In implementation, such sensors would, for example, consist of Inertia Measurement Units (IMUs), GNSS sensors, Lidars and/or other positioning sensors.

B. Implementation

Within this section, the implementation of the different elements within Matlab is discussed. The Matlab code has been uploaded to a shared GitHub for further references. [5]

1) *Controlling drone movements*: Two types of flights modes were implemented in the Matlab model - autonomous following the pre-determined trajectory and manually controlled by user. First type of the flight assumes calculating 3D trajectory based on the position coordinates and longitudinal speed. The drone is supposed to perform avoidance movements

while flying accordingly to the trajectory. The second option is controlling the drone manually to check if it is possible to control the drone efficiently.

a) *User Input Controller*: The system is prepared to be used with Radiomaster Pocket. *Formatting* block has descriptions (eg. SA, SE) which are names of the buttons of the radio controller. For controlling the drone, sticks are used, but a button was also set to activate the large avoidance scenario. In reality this would be triggered by a sensor as a human operator would not be fast enough. SE single press button switch was chosen because the function is supposed to act like an instant, temporary action and the system should come to the normal behavior right after releasing the switch.

Formatting block was configured to test two types of flight modes - angle and acro. In the figure 2 system for angle mode is presented, using the manual switches it can be set to acro. The only difference is how pitch and roll behavior is controlled. In acro mode, the controller needs to control the orientation. The drone will stay in the same orientation even if the sticks are centered again. To achieve that, integration on roll and pitch axes was performed.

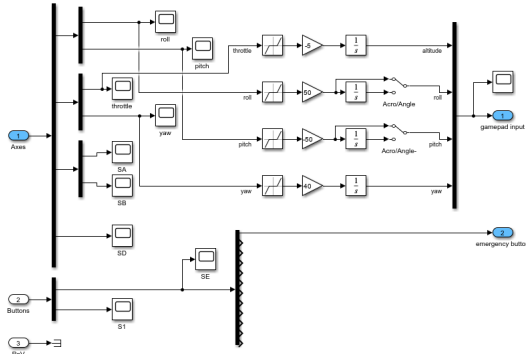


Figure 2. Formatting block of the radio controller input option.

b) *Pre-determined Trajectory*: To be able to simulate autonomous drones, the trajectory based on the data determined by user is created. As an input, waypoint coordinates and path "speeds" are set. Also the parameter ρ_d is set which determines how fast the drone will follow the trajectory. A matrix is constructed to capture both the boundary conditions at the start and end of each segment, as well as the necessary continuity constraints (position, velocity, and acceleration) between the neighboring segments. For example, first row determines position, second row velocity at the start of the first segment. Forth to sixth rows includes data about continuity between first and second segment. As an example the coefficient matrix for three segment trajectory is

shown 1.

$$\begin{bmatrix} b(0)' & O & O \\ b_{\text{prime}}(0)' & O & O \\ b(1)' & O & O \\ b_{\text{prime}}(1)' & -b_{\text{prime}}(1)' & O \\ b_{\text{pprime}}(1)' & -b_{\text{pprime}}(1)' & O \\ O & b(1)' & O \\ O & b(2)' & O \\ O & b_{\text{prime}}(2)' & -b_{\text{prime}}(2)' \\ O & b_{\text{pprime}}(2)' & -b_{\text{pprime}}(2)' \\ O & O & b(2)' \\ O & O & b(3)' \\ O & O & b_{\text{prime}}(3)' \end{bmatrix} \quad (1)$$

To define spline segments, coefficient vectors α_x , α_y and α_z are obtained. The final 3D path, shown in figure 3, is a combination of the paths computed using a matrix of waypoints, where each column represents a point in the 3D space.

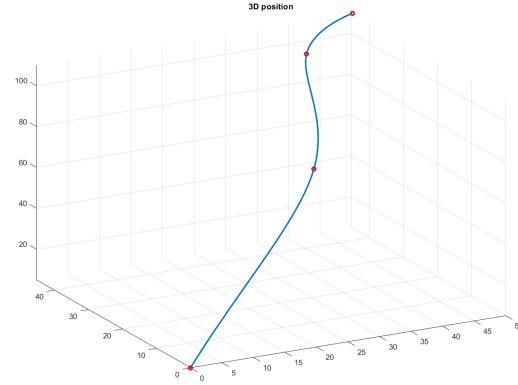


Figure 3. Example of the trajectory.

Calculated data is used to create a trajectory. User can also adjust the sigma value which changes how many points between specific waypoints are created. Finer sampling (smaller intervals between sigma values) means more points are used to define the spline.

In the Simulink all the prepared data is transferred to *Desired Trajectory* subsystem block where it is combined and the actual trajectory is created. It is later send to *Outer loop position control*.

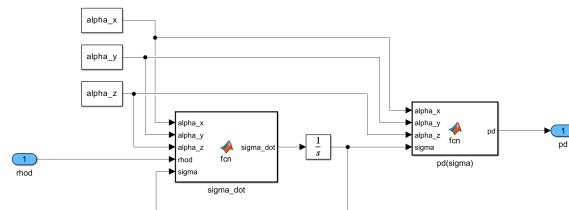


Figure 4. Desired trajectory subsystem

2) *Avoidance System*: The Avoidance System is split into a few parts. The architecture of the system can be seen in the figure 5. The two different systems each have their own role. The Pseudo-random-movement will create a constant distraction during normal flight. While the Heavy Avoidance will respond to larger threats.

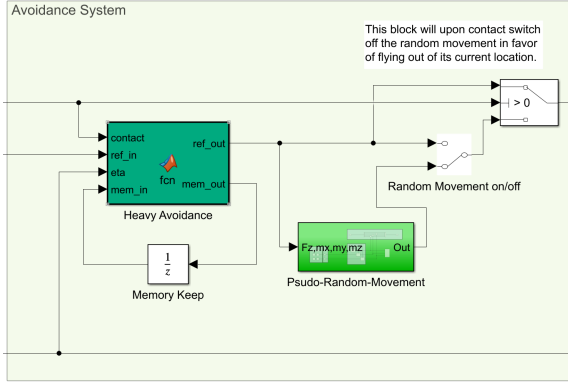


Figure 5. Avoidance System

a) *Pseudo-Random-Movement*: The Pseudo Random motion is generated within the *Pseudo-Random-Movement* subsystem block. This behavior can be switched on and off before running the whole simulation. The content of this block has 3 sub parts. First is the generation of the number. Since this is generated through a computer algorithm (Random Number block) it is not considered fully random. It is key to apply different 'seed' values to the block to make sure they do not follow the same values. To be sure that the pseudo randomly generated value (being the additional movement angle) is added on top of current angle, it is held using a sample hold block with a pulse clock. The output of this is added to the input signal. The effect being that the drone is pushed around in these random direction.

b) *Heavy Avoidance*: The Heavy Avoidance block has the action to make the drone fly away from a threat as fast as possible. The drone does so by receiving a signal, from a sensor, or in this case a user-input. Once the signal is received, the drone will stop following its trajectory or basic controls, and instead focus on flying out of the current location. If the altitude is high enough, the drone could move sideways with its maximum tilt. However, if the drone is at a low altitude, this could result in crashes, thus gaining altitude would be a safer method. To differentiate between these scenarios, the following code was implemented within the Matlab function block:

```
function [ref_out,mem_out]= fcn(contact,
    ref_in,eta,mem_in)
if contact == 0
    ref_out = ref_in;
    mem_out = [-1;eta(3)];
```

```
else
    if mem_in(1) == 0
        ref_out = [mem_in(2)-150;0;0;0]; %
            Shoot up and away (150m away)!
        mem_out = [0;mem_in(2)];
    elseif mem_in(1) == 1
        ref_out = [mem_in(2);50;50;0];% Stay
            at the same height but move
            away
        if -1*eta(3) < (-1*mem_in(2)-50) %
            Dont drop more than 50m
            ref_out = [mem_in(2);0;0;0];
        end
        mem_out = [1;mem_in(2)];
    else
        ref_out = ref_in;
        mem_out = [-1;mem_in(2)];
    end

    % Contact Detected:
    % Test if altitude is high enough for
    % side-drop. Else move up away!
    if mem_in(1) == -1 % Only test once.
        if -1* eta(3) < 150
            mem_out = [0;eta(3)]; %Too Low
        else
            mem_out = [1;eta(3)]; %High
            Enough.
        end
    end
end
end
```

By keeping the memory of its activation altitude, the drone is able to know what reaction mode it activated in, making sure that when the drone reaches the altitude in which the reaction would change, the drone does not start acting irrational, but stick to its initial plan. By using its current height, it is also able to ensure the drone does not travel into a lower airspace than intended.

3) *Feedback Controller*: This part of the system is responsible for controlling the drone towards the desired orientation required for the motion. Within the current designed setup, the controller is run by a PID Controller. This controller takes the current orientation data (η) and the desired position to create the error which then is fed into the PID blocks. These will create a proportional, integral and derivative solution and feed this into the drone motion controller. These controllers require further adjustments when placing it within a test setup, however the precise tuning of the PID is outside the scope of this project.

4) *Drone Motion Controller*: The drone motion controller part will take the knowledge of the drones motor system and implement the thrust of each motor based on the desired motion outputted by the PID. This part is often referred to as the Motor Mixing Algorithm (MMA).

The MMA returns the propulsion (τ_p) which is used in the model of the drone shown in equation 2.

$$\begin{cases} \dot{\eta} = \mathbf{J}_{\Theta}(\eta)\nu \\ \mathbf{M}_{RB}\dot{\nu} + \mathbf{C}_{RB}(\nu)\nu = \tau_g + \tau_p \end{cases} \quad (2)$$

The model of the drone is fed into a VR-sink, which displays the virtual environment for the user. As can be seen in figure 6

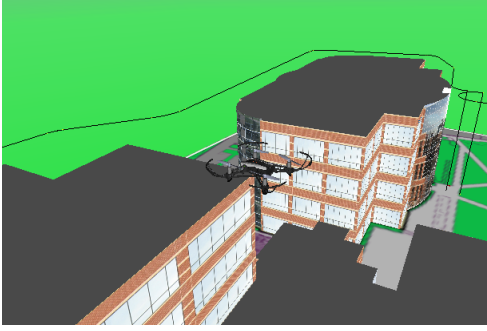


Figure 6. Screen capture of the Virtual Environment.

IV. RESULTS

A. Pseudo Random Movement

Figures 7 and 8 show how the position of the drone during the flight changes. The drone performing demo flights was supposed to move from 0, 0, 0 point to the 50m, 45m, 110m. The flight without randomness was stable and smooth. The drone stabilized at the desired position. The second try with the randomness enabled (8) was much more "chaotic". Still the expected position was reached but the drone had random oscillations. Moreover, even when it was at the final position, it was still moving around. This kind of behavior makes it unpredictable.

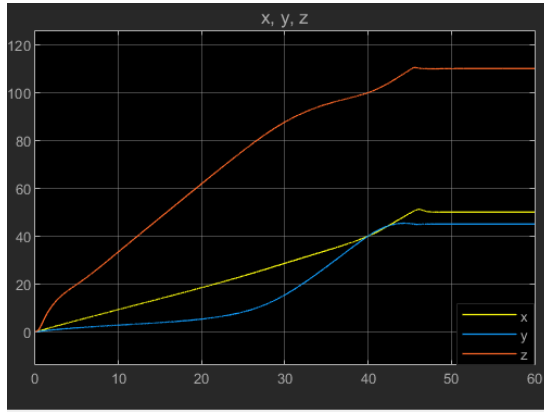


Figure 7. Drone position during the flight without the Pseudo Random modifier

B. Heavy Avoidance

As it was described in paragraph III-B2b two types of heavy avoidance procedures were implemented. In figure 9, the drone was at approximately 75m altitude. This is considered low altitude. Being below 150m the 'shooting up scenario' is activated. The drone immediately gained 150m and stabilized at the 225m altitude. When the alarm was canceled (40th second

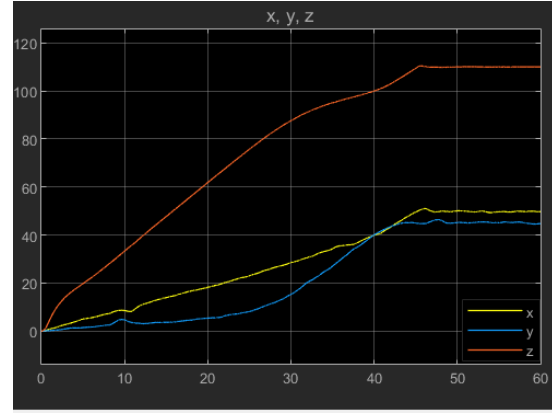


Figure 8. Drone position during the flight with the Pseudo Random modifier

of simulation), after which the drone started to follow the trajectory again. Due to the large drop oscillations occurred in the position afterwards.

In figure 10, the alarm was raised at the altitude of 200m. It means that the drone was high enough to be able to escape sideways. X and Y position suddenly changed and the altitude was stable.

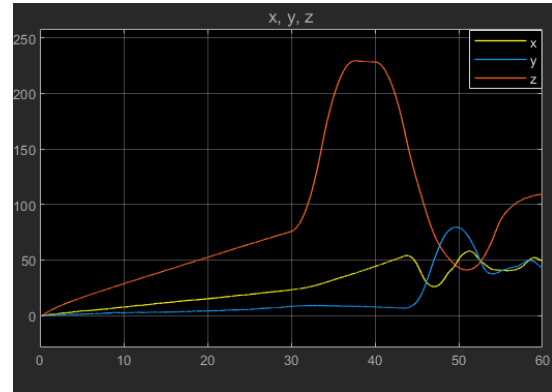


Figure 9. Heavy avoidance procedure at low altitude.

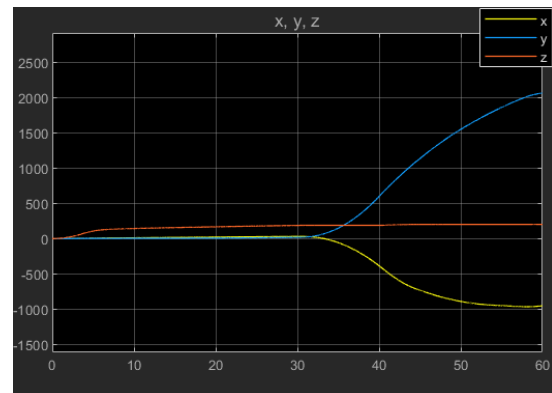


Figure 10. Heavy avoidance procedure at high altitude.

V. DISCUSSION

As can be seen within the result section above, the drone does indeed perform evasive maneuvers. Though it must be clearly stated that during the testing, a multitude of issues arose. Although the results show that the implementation of such system in drones is not only possible it is also something that should be considered when creating defense systems. Looking at the different parts separately, a multitude of improvements can be identified. More tuning needs to be performed. The drone was well tuned and flyable with the Pseudo Random input enabled but Heavy Avoidance procedures turned out to be too rapid and extreme. As it can be seen in the graphs presented in section IV, some motions created oscillations due to tuning of the PID or extreme reactions due to lack of additional forces. For example sideways escape results in sudden change of drone position in Y axis from 0 to about 2000m. Although this is the aimed result, 2km distance would be unrealistic in such a time frame. This phenomenon could be due to the lack of air resistance.

Talking about how the drone behaves with the randomness enabled, when it follows the pre-defined trajectory, it is quite stable. Graphs shows that everything goes off accordingly to the plan. However, controlling the drone manually using the RC controller is challenging. The pilot cannot react as fast and precise as the feedback controller. It means that the piloted drone moves around even more. Theoretically it makes it even harder to shoot down but in certain situations, when it is necessary to fly close the buildings etc. it can be dangerous. It is recommended to adjust the Intensity Parameters accordingly to how skilled the pilot is and what the scenario of flight is.

The developed system can potentially be implemented in drones created by Carmenta which was present at International Drone Show 2024. They want to integrate autonomous drones in combat vehicles (CV90). The drones are supposed to be used as an additional source of information in the battlefield. Randomness of movements and heavy avoidance procedures would definitely increase their survivability.

A. Future Work

As can be seen from other work, quaternions provide much more agility with drones.[6] During the testing the limits of the Euler-PID system were found where the simulation would crash. By implementing a quaternion system it would be possible to create more extreme motions, allowing the Heavy-Avoidance to be more robust.

The original concept was to have trajectory calculation scalable. The user would determine only waypoint-coordinates and velocities. The code was prepared (part of it below) but due to issues with the automatic creation of the coefficient matrix the idea was dropped.

```
points = cell(1, numPoints);
for i = 1:numPoints
    point = eval(sprintf('point%d', i));
    points{i} = point;
end
velocities = cell(1, numPoints);

for i = 1:numPoints-1
    v = compute_normalized_difference(points
        , i, i + 1);
    velocities{i}(1) = v(1)*vel(i);
    velocities{i}(2) = v(2)*vel(i);
    velocities{i}(3) = v(3)*vel(i);
end
velocities{numPoints} = vel(numPoints);

function normalized_diff =
    compute_normalized_difference(points,
        index1, index2)
    p0 = points{index1};
    p1 = points{index2};
    difference = p1 - p0;
    normalized_diff = difference / norm(
        difference);
end
```

VI. CONCLUSION

Generally the paper shows how the implementation of a small and large caliber avoidance system can be designed and deployed on a drone, using a simulation. With these results, it can become clear how such a system could potentially be a threat to modern defense systems, and how this can be improved. To make this implementation even more successful a few things were found that the system could benefit from. For instance using a different controlling system, so instead of using Euler, use Quaternions, to allow for even more rapid and unpredictable movements. This all highlights how defense systems of tomorrow need to keep striving to create more modern system, able to predict for the upcoming changes.

REFERENCES

- [1] T. G.-. O. E. Enterprise, "The red team handbook."
- [2] D. Kunertova, "The war in ukraine shows the game-changing effect of drones depends on the game," *Bulletin of the Atomic Scientists*, vol. 79, no. 2, pp. 95–102, 2023.
- [3] R. A. D. AG, "Oerlikon Revolver Gun@Mk3," 2022.
- [4] J.-Y. Lee, B.-U. Jo, G.-H. Moon, M.-J. Tahk, and J. Ahn, "Intercept point prediction of ballistic missile defense using neural network learning," *International Journal of Aeronautical and Space Sciences*, vol. 21, no. 4, pp. 1092–1104, 2020.
- [5] A. Kolodziej and T. van der Sterren, "Gnc automatic evasive maneuver generation for remotely-operated drones," https://github.com/u0000000/GNC-Automatic_evasive_maneuver_generation_for_remotely-operated_drones, 2024. Accessed: 2024-05-01.
- [6] J. Cariño Escobar, H. Abaunza González, and P. Castillo García, "Quadrotor quaternion control," 06 2015.